

高效验证的跨链状态证明方案

孙文拓^①

^① (中国科学技术大学网络空间安全学院 合肥 230026)

摘要: 随着区块链生态的不断发展,跨链已经成为区块链基础设施的核心需求。但是不同区块链之间架构互异且互不信任,若要完成跨链操作需要跨链状态证明来向目标链证明其他区块链上特定状态的真实性与完整性。基于 zk-SNARK 的跨链状态证明方案因其无需可信第三方且高效验证的特性,十分契合计算资源受限的区块链场景。然而跨链状态证明电路中涉及大量哈希验证,电路规模庞大,直接生成证明存在严重的效率问题。为解决此瓶颈,近年来提出了分布式零知识证明技术,通过电路分割和并行计算大大提高了证明生成效率,部分方案实现了的常数级证明大小和验证复杂度。但是实验证明在以以太坊中单个证明验证需消耗超过 60 万 Gas,且在实际跨链场景中同一时间窗口内往往存在大量并发的状态证明需要验证,逐个验证则进一步增加了区块链计算压力。本文在已有分布式零知识证明协议的基础上提出了高效验证的跨链状态证明方案。一方面,利用多点聚合技术将单个证明的大小和验证复杂度进一步降低,为证明批量聚合提供了基础。另一方面,利用 KZG 承诺的加法同态性通过挑战协商实现了证明的批量聚合,进一步降低了证明的验证开销。理论和实验分析表明,本方案在没有损失安全性的前提下,单个证明大小减少约 52.6%,gas 开销节省 37.2%,同时聚合证明实现了约 60%的链上 Gas 开销节省。

关键词: 区块链; 跨链; 零知识证明; 分布式零知识证明

中图分类号: [单击此处键入中图分类号] **文献标识码:** A **文章编号:**

DOI: 10.11999/JEIT××××××

1 引言

随着区块链技术的成熟,不仅公有链的发展如火如荼,不同行业也都部署了私有链或联盟链来对其数据进行全流程管理和背书。然而,不同区块链系统之间架构各异且互不信任,互通困难,形成了大量数据孤岛,跨链互操作已经成为区块链基础设施的核心需求之一。针对跨链操作困难的问题,跨链委员会[1][2]将来自不同区块链的多个验证节点组成委员会对跨链状态进行集体投票背书,但其中心化程度较高,更容易被攻击者选择成为破坏整个系统的突破口。另一方面,其在安全性与效率之间存在冲突,若委员会规模较小,则跨链背书生成效率高但安全性低容易被攻破,若规模较大则会降低状态背书效率。中继链[3][4]通过构建专用的中继区块作为各个区块链之间沟通的桥梁链维护各链状态,但需要额外建立共识机制,增加了系统复杂度。此外系统跨链性能受中继链吞吐量影响,后续系统扩容升级的灵活性不足。

基于零知识证明的跨链状态证明是近年来最新的发展方向,其核心优势为无可信第三方,无额外隐私泄露和高效验证。协议中仅包含证明者和验证者两种角色,证明者能够在不向验证者透露额外隐私数据的前提下,通过数学手段使验证者确信某个计算状态转移的正确性。现广泛应用的成熟 zk-SNARK 协议如 Groth16[5]和 Plonk[6]的证明大小固定且证明验证时间仅为毫秒量级。这种非交互、低带宽占用且隐私保护的特性,使 zk-SNARK 协议在区块链扩容和隐私保护的方面有着广泛应用。在扩容方面,为提高区块链吞吐量,基于 zk-SNARK 为大量链下交易生成正确性证明,链上智能合约无需重复执行交易流程,仅需验证证明的正确性即可完成状态更新,如 zkRollup[7]等。在隐私保护方面,利用 zk-SNARK 的零知识性,保护链上交易信息如交易金额,交易双方身份信息,如最早应用 zk-SNARK 到加密货币领域的 ZeroCash[8]等。

在跨链状态证明场景中直接使用 zk-SNARK 协议完成证明,存在明显的效率问题。跨链状态证明要求目标链能够完整验证原区块链中的状态存储逻辑从而验证相关状态的真实性与完整性,而区块链系统大多采用默克尔树的某种变体如默克尔哈希树或默克尔前缀树等进行状态数据存储,该类数据结构具有维护简便、更新高效且支持动态扩容的优势,但其内置的哈希函数多包含大量非线性运算,这类运算在零知识证明电路中会产生大规模的电路约束,同时为实现全量状态存储,默克尔树的平均深度通常在 10-20 层之间,且随着链上状态量的增加,深度可能进一步提升。因此跨链状态证明的电路规模通常很大,且证明生

收稿日期: ××××-××-××; 改回日期: ××××-××-××; 网络出版: ××××-××-××

*通信作者: 李明 liming123@aircas.ac.cn (通信和一作均需唯一)

基金项目: 基金 1, 基金 2, 基金 3 (国防科工、军事、装备预研等基金不要注明)

Foundation Item: Item1, Item2, Item3

成过程涉及大量的多项式运算，因此需要利用快速傅里叶变换（Fast Fourier Transform, FFT）和多标量乘法（Multi-Scalar Multiplication, MSM），计算开销极大，并很容易触发内存溢出。针对这一瓶颈，Wu 等人^[9]最早提出了分布式零知识证明方案，这类方案的基本思路是把大规模电路拆分为多个子电路，在多个子计算节点中并行执行计算，从而实现证明生成的线性扩展。在此基础上，现有的分布式方案^{[9][10]}主要聚焦于优化证明生成时间和子节点间的通信开销，在一定程度上忽视了证明的验证开销在区块链环境下的敏感性。尽管 Pianist 方案将证明验证复杂度降低为常数级，但实验结果证明在以太坊智能合约中对该方案的单词证明验证即需消耗超过 60 万 Gas。另一方面，实际跨链场景中同一时间窗口内往往存在大量并发的状态证明请求，逐个验证将进一步占用大量计算资源，制约了零知识证明技术在跨链场景中的实际部署。

为了解决上述问题，本文在保留 Pianist 分布式零知识证明架构的线性扩展性和低通信量优势的前提下，提出了证明压缩和证明聚合两个方案，进一步降低了跨链通信开销和链上验证成本，本文的主要贡献总结如下：

(1) 针对单个跨链状态证明验证开销过高的问题，我们提出了基于多点聚合的证明压缩方案将分布式证明框架中多个独立的多项式开启证明聚合为常数个，在不增加证明生成渐近复杂度的前提下，减少了所需承诺和开启证明的数量，从而降低了单个跨链状态证明的大小和链上验证开销。

(2) 针对跨链场景中多个并发状态证明需逐个验证导致开销线性增长的问题，我们提出了证明批量聚合方案。该方案利用 KZG 承诺的加法同态性，通过安全的挑战协商机制使证明之间共享验证参数，将批量验证所需的配对运算次数从随证明次数线性增长降至常数次，从而降低了跨链证明的通信复杂度和验证开销。

(3) 我们形式化证明了证明压缩和批量聚合机制在代数群模型和随机预言机模型下满足完备性、知识可靠性和零知识性，安全性归约到底层 Pianist 协议及 q-DLOG 困难假设，不引入额外的安全假设。理论分析和实验结果表明，方案在保持证明生成时间复杂度和通信复杂度与 Pianist 协议一致的前提下，单个证明大小减少约 52.6%，Gas 开销节省 37.2%，批量聚合在典型场景下实现了约 60% 的链上 Gas 节省。

2 相关工作

针对跨链状态证明这类超大规模电路的证明生成效率低的问题，学术界提出了一系列分布式零知识证明方案。最早的是 Wu 等人提出的 DIZK^[9]通过分布式执行 FFT 和 MSM，成功将证明的电路规模扩展到了数十亿门。然而其局限性在于通信复杂度较高，与电路规模 N 呈线性关系 $O(N)$ 。为了克服通信瓶颈，Xie 等人在 zkBridge^[10]中提出了 deVirgo，其以无需可信设置的 Virgo 协议^[11]为基础，设计了分布式 sumcheck 协议与聚合多项式承诺方案，既实现了证明生成的线性加速，又避免了证明大小随子电路数量线性增长的弊端。然而，deVirgo 在优化证明端并行效率的同时，未能原生适配链上验证场景，其基于默克尔树与 FFT 计算的多项式承诺生成的原始证明大小约 210KB，链上直接验证需消耗近 80M gas，远超区块链单块 Gas 限制，无法直接部署于智能合约。为了实现链上的高效验证，不得不使用 Groth16 在 deVirgo 的基础上进行了一次递归证明，这虽然解决了链上验证开销问题，但增加了证明生成时间和复杂度。Liu 等人提出了基于 Plonk 的 Pianist 协议^[12]，将单变量多项式扩展成双变量，子节点仅负责对 X 轴多项式计算承诺和证明，主节点收到对应子电路的承诺和证明后，再对 Y 轴多项式进行承诺和证明，并针对性地将原 Plonk 协议中的 KZG 承诺^[13]转换为分布式 KZG 承诺，最终实现了常数级的证明大小和常数级的验证时间。Hekaton^[14]通过高效全局内存技术处理子电路间共享线，支持任意计算且水平扩展性强，能高效适配可验证密钥目录和 RAM 计算等实际应用。其不足之处在于证明过程仍存在准线性成本，且证明大小和验证时间随子电路数量呈对数关系。HyperPianist^[15]是基于 HyperPlonk^[16]的分布式零知识证明，将布尔超立方体的 d 多变量多项式转移到分布式场景，重新设计了分布式 sumcheck 协议和 deDory 分布式多项式承诺方案，实现了线性时间的证明生成效率，但其验证开销退化为对数级。最终我们选择了证明大小和验证开销均为常数的 Pianist 协议作为基础进行进一步验证开销上的优化。

针对 zk-SNARK 协议的验证开销优化，现有工作主要包括三类技术路线。第一类是递归证明技术 [17]，其核心思想是将一个 zk-SNARK 证明的验证逻辑编码为算术电路，用外层证明来验证内层证明的正确性，从而将多个证明的验证压缩为对单个递归证明的验证。该技术天然适用于 zkRollup 等需要持续积累大量来自不同用户独立交易证明的场景，此外 deVirgo[10]最后的链上验证环节也通过 Groth16 对其原始证明进行递归压缩以适配链上验证。然而，递归证明要求为内层验证算法设计专用的算术电路，电路复杂度较高，且每一层递归都引入额外的证明生成开销。在分布式跨链状态证明场景中，同一批证明由相同的电路结构并行生成，具有高度的结构一致性，采用递归方式逐层嵌套不仅浪费了这一结构优势，还显著增加了证明生成延迟和系统复杂度。第二类是证明聚合技术，Gailly 等人提出的 SnarkPack[18]利用 Groth16 证明中配对乘积结构的代数性质，通过内积论证将多个 Groth16 证明聚合为单个证明进行批量验证，适用于 Filecoin 等需要批量验证大量独立 Groth16 证明的场景。但 SnarkPack 的聚合机制深度依赖 Groth16 证明的特定三元组结构 (A_i, B_i, C_i) ，而基于 Plonk 的分布式证明框架采用多项式承诺和开启证明的验证范式，不具备兼容的代数接口，因此难以直接适配。第三类是多项式承诺的开启聚合技术，Boneh 等人提出的 Shplonk[19]针对 KZG 多项式承诺方案，通过构造聚合商多项式将多个多项式在多个不同挑战点的开启证明合并为单次配对验证，减少了承诺个数和配对运算次数。该技术直接作用于多项式承诺层，与上层证明系统的具体协议结构解耦，因此能够自然地嵌入基于 Plonk 的分布式证明框架。然而，Shplonk 仅解决了单个证明内部多个开启证明的聚合问题，未涉及多个独立证明之间的批量聚合。综上，上述三类工作各自针对特定场景解决了验证优化的某一方面，但尚未有方案在分布式证明框架下同时实现单证明压缩与多证明批量聚合，这正是本文要解决的核心问题。

3 高效验证的跨链状态证明方案

本文以 Pianist 协议[12]的双变量多项式的分布式 Plonk 协议框架为基础，分别提出证明压缩和证明聚合两项优化机制来优化证明验证效率。其中证明压缩专注于单个证明的验证优化，通过多点聚合技术减少开启证明数量和优化配对运算，证明聚合针对同一时间窗口内多个并发的跨链状态证明请求，通过将多个独立证明聚合为单个批量证明，进一步降低通信复杂度和链上验证开销。本节首先概述 Pianist 协议并分析其在验证阶段存在的效率瓶颈，随后分别给出证明压缩和证明聚合的具体构造。

3.1 Pianist 协议概述

Pianist 将原始 Plonk 协议[6]的单变量约束系统扩展为双变量形式，其中子节点负责 X 轴证明生成，主节点聚合 X 轴计算结果后对 Y 轴生成证明。具体而言，给定包含 M 个子电路的总电路，每个子节点 $\mathcal{P}_i$ 持有规模为 $T = N/M$ 的子电路 C_i 并在本地生成对应的见证多项式 $a_i(X), b_i(X), o_i(X)$ 、置换累加多项式 $z_i(X)$ 和商多项式 $h_i(X)$ ，这些单变量多项式沿用了 Plonk 协议中门约束和复制约束的定义。主节点 $\mathcal{P}_0$ 则通过 Y 轴拉格朗日基函数 $R_i(Y)$ 将各子节点的局部多项式聚合为全局二元多项式，例如 $A(Y, X) = \sum_{i=0}^{M-1} R_i(Y) a_i(X)$ ，其余多项式同理。最终， M 个子电路的约束被聚合为统一的二元恒等式 $G(Y, X) + \lambda P_0(Y, X) + \lambda^2 P_1(Y, X) = V_X(X) H_X(Y, X) + V_Y(Y) H_Y(Y, X)$ ，其中 λ 为门约束与复制约束的随机聚合挑战， $V_X(X) = X^T - 1$ 和 $V_Y(Y) = Y^M - 1$ 分别为 X 轴和 Y 轴的消失多项式， $H_X(Y, X)$ 和 $H_Y(Y, X)$ 为对应的商多项式。验证 M 个子电路的正确性等价于验证该二元恒等式成立。

为支持上述双变量构造，Pianist 设计了分布式 KZG 承诺方案 DKZG。子节点 $\mathcal{P}_i$ 通过预分发的公共参数 $\mathbf{U} = (g^{R_i(r_Y)L_j(r_X)})_{i,j}$ 在本地计算各局部多项式的承诺，主节点利用 KZG 的加法同态性将各子节点承诺聚合为全局二元多项式承诺，例如 $\text{com}_A = \prod_{i=0}^{M-1} \text{com}_{a_i}$ 。整个证明流程遵循“承诺-挑战-开启”的三阶段结构，子节点首先生成见证多项式承诺并由主节点聚合，主节点经 Fiat-Shamir 变换派生挑战 η, γ, λ 后，子节点继续生成置换累加多项式 $Z(Y, X)$ 和 X 轴商多项式 $H_X(Y, X)$ 的承诺。随后主节点派生 X 轴挑战 α ，各子节点计算本地多项式在 α 处的取值并生成 DKZG 第一层开启证明，主节点据此聚合得到 Y 方向的求值多项式 $A(Y, \alpha)$ ， $B(Y, \alpha)$ ， $O(Y, \alpha)$ ， $Z(Y, \alpha)$ ， $H_X(Y, \alpha)$ ，和 $Z(Y, \omega_X \alpha)$ 。最后，主节点派生 Y 轴挑战 β ，对上述 Y 方向多项式在 β 处求值并生成 DKZG 第二层开启证明，得到包含见证、置换、商多项式承诺与两层

开启证明的最终证明 π ，其总大小约为 $27|\mathbb{G}_1|+15|\mathbb{F}|$ ，在常数级范围内，与电路规模 N 和子节点数量 M 无关。

由上述流程可见，Pianist 的证明大小为 $27|\mathbb{G}_1|+15|\mathbb{F}|+1$ ，虽与电路规模 N 和子节点数 M 无关，但其中包含了大量独立的开启证明，每个待打开的多项式需在 X 轴和 Y 轴两个维度分别生成独立的 DKZG 开启证明，且置换多项式 $Z(Y, X)$ 需在 α 和 $\omega_X \alpha$ 两个不同的 X 轴挑战点打开，进一步增加了开启证明数量。这些独立的开启证明使得上链证明包含约 27 个 $\mathbb{G}_1$ 群元素和 15 个域元素，而证明大小直接决定了跨链通信的带宽占用，同时实际跨链场景中同一时间窗口内往往存在多个并发的状态证明请求，通信开销将随证明数量线性增长。此外，当目标链为以太坊等公有链或基于 EVM 的联盟链、私有链时，证明作为 calldata 传入验证合约，其大小直接转化为 Gas 消耗，实验表明单次验证即需消耗超过 60 万 Gas，多个证明的逐一验证将进一步加剧链上资源压力。因此，减少开启证明数量以压缩证明大小，是降低链上验证成本的关键，也是后续证明压缩的核心优化目标。

3.2 证明压缩

针对 Pianist 协议中单个证明验证开销过大的问题，我们提出了证明压缩方案，沿 X 轴和 Y 轴两个维度分别设计聚合算法， X 轴维度将多个挑战点处的多项式开启合并为单个商多项式承诺 com_{Q_X} ， Y 轴维度将多个一元多项式的开启折叠为单个聚合开启证明 $\pi_{1,\text{agg}}$ 。两层聚合完成后， X 轴验证需 3 次配对， Y 轴验证需 2 次配对，两者仍为相互独立的验证等式。为使后续的批量聚合能够对多个证明的验证等式执行统一的随机线性组合，我们进一步引入随机数 ρ 将两组等式合并为单个四配对等式 $e(\text{com}_{Q_X}, [Z_T(\tau_X)]_2) \cdot e(\pi_{1,\text{agg}}, g_2^{\tau_Y - \beta})^\rho = e(D_{\text{lin}}, g_2) \cdot e(D_r, g_2^{\tau_X})$ ，其中 $Z_T(X) = (X - \alpha)(X - \omega_X \alpha)$ ， D_{lin} 和 D_r 为由聚合商承诺、聚合开启证明及各多项式在挑战点取值构造的线性组合量，具体构造见算法 1。

算法 1: 证明压缩 ProofCompress

输入：全局承诺 $\text{com}_A, \text{com}_B, \text{com}_O, \text{com}_Z, \text{com}_{H_X}$ ，挑战 η, γ, λ ，子节点 $\mathcal{P}_i$ 持有局部多项式 $a_i, b_i, o_i, z_i, h_{X,i}$

输出：压缩证明 π_c

// 主节点 $\mathcal{P}_0$ 执行:

1: $\alpha \leftarrow H(\text{com}_A, \text{com}_B, \text{com}_O, \text{com}_Z, \text{com}_{H_X})$ ， $\nu \leftarrow H(\alpha)$ ，将 α, ν 分发至各子节点 $\mathcal{P}_i$

// 各子节点 $\mathcal{P}_i$ ($i = 0, \dots, M-1$) 并行执行:

2: $Q_{X,i}(X) \leftarrow \sum_{S \in \mathcal{S}_\alpha} \nu^{\text{idx}(S)} \cdot \frac{S_i(X) - S_i(\alpha)}{X - \alpha} + \nu^{\text{idx}(Z')} \cdot \frac{Z_i(X) - Z_i(\omega_X \alpha)}{X - \omega_X \alpha}$ ， $\text{com}_{Q_{X,i}} \leftarrow g^{R_i(\tau_Y) \cdot Q_{X,i}(\tau_X)}$

3: 将 $\{S_i(\alpha)\}_{S \in \mathcal{S}_\alpha}, Z_i(\omega_X \alpha), \text{com}_{Q_{X,i}}$ 发送至 $\mathcal{P}_0$

// 主节点 $\mathcal{P}_0$ 执行:

4: $\text{com}_{Q_X} \leftarrow \prod_{i=0}^{M-1} \text{com}_{Q_{X,i}}$

5: foreach $S \in \mathcal{S}_\alpha$: $v_S(Y) \leftarrow \sum_{i=0}^{M-1} R_i(Y) \cdot S_i(\alpha)$

6: $v_{Z'}(Y) \leftarrow \sum_{i=0}^{M-1} R_i(Y) \cdot Z_i(\omega_X \alpha)$ ， $H_Y(Y, \alpha) \leftarrow (G(Y, \alpha) + \lambda P_0(Y, \alpha) + \lambda^2 P_1(Y, \alpha) - V_X(\alpha) H_X(Y, \alpha)) / V_Y(Y)$

7: $\beta \leftarrow H(\alpha, \nu, \text{com}_{Q_X})$ ， $\mu \leftarrow H(\beta)$

8: $G_Y(Y) \leftarrow \sum_{P \in \mathcal{V}_Y} \mu^{\text{idx}(P)} \cdot P(Y)$ 其中 $\mathcal{V}_Y = \{v_S(Y) \mid S \in \mathcal{S}_\alpha\} \cup \{v_{Z'}(Y), H_Y(Y, \alpha)\}$

9: $Q_Y(Y) \leftarrow (G_Y(Y) - G_Y(\beta)) / (Y - \beta)$

10: $\pi_{1,\text{agg}} \leftarrow g^{Q_Y(\tau_Y)}$ ， $\text{com}_{G_Y} \leftarrow g^{G_Y(\tau_Y)}$

11: foreach $S \in \mathcal{S}_\alpha$: $S(\beta, \alpha) \leftarrow v_S(\beta)$

12: $Z(\beta, \omega_X \alpha) \leftarrow v_{Z'}(\beta)$ ； $H_Y(\beta, \alpha) \leftarrow H_Y(Y, \alpha). \text{Eval}(\beta)$

13: 返回 $\pi_c \leftarrow (\{\text{com}_A, \text{com}_B, \text{com}_O, \text{com}_Z, \text{com}_{H_X}, \text{com}_{Q_X}, \text{com}_{G_Y}\}, \pi_{1,\text{agg}}, \{S(\beta, \alpha)\}_{S \in \mathcal{S}_\alpha} \cup \{Z(\beta, \omega_X \alpha), H_Y(\beta, \alpha)\})$

如算法 1 所示，证明压缩流程包含挑战派生、分布式计算和全局聚合三个阶段。首先主节点对所有全

局承诺取哈希派生 X 轴挑战 α 和聚合随机数 ν ，并将其分发至各子节点，为后续的多点聚合提供公共挑战基础，对应算法第 1 行。其次，各个子节点在本地并行构造 X 轴聚合商多项式 $Q_{X,i}(X)$ 。子节点在计算 $Q_{X,i}(X)$ 的 KZG 承诺后，连同本地多项式在挑战点的求值一并发送至主节点，对应算法 2-3 行。最后，主节点执行全局聚合，先利用 KZG 承诺的加法同态性将各子节点的局部商承诺聚合为全局 X 轴商承诺 com_{Q_X} ，对应算法第 4 行。随后在 Y 轴维度上，将所有待验证的一元多项式通过随机数 μ 组合为聚合多项式 $G_Y(Y)$ ，并构造对应的商多项式 $Q_Y(Y)$ 生成聚合开启证明 $\pi_{1,\text{agg}}$ ，对应算法 5-10 行。最终将全局承诺、聚合开启证明和各多项式求值组装为压缩证明 π_c 输出，对应算法 11-13 行。

证明的验证流程与 Pianist 协议基本一致，验证者通过 Fiat-Shamir 变换重算所有挑战值后检查代数约束等式，区别仅在于本方案利用随机数将两组配对验证等式合并为单个四配对等式。由于 X 轴和 Y 轴的多点聚合将多个独立的开启证明分别压缩为单个商承诺和单个聚合开启证明，上链证明所含的群元素数量从约 27 个减少至 12 个，降低了跨链时的通信开销和链上验证开销，详细分析见 6.1 节。

3.3 证明聚合

证明压缩在单个证明维度上降低了验证开销，但实际跨链场景中同一源链至目的链方向存在多个并发的状态证明请求，若对每个证明独立验证，通信开销和链上验证开销将与证明数量 K 线性相关，限制了跨链状态证明的应用。为此，我们提出证明聚合方案将 K 个证明 $\{\pi_i\}_{i=1}^K$ 压缩为一个批量证明 π_{batch} 。

证明聚合的基础是利用 KZG 承诺的加法同态性将多个承诺聚合成一个，而聚合的前提是所有 KZG 承诺共享相同的挑战点 α 和 β ，这是由配对合并等式中的 $\mathbb{G}_2$ 元素 $[Z_T(\tau_X)]_2$ 、 $g_2^{\tau_Y - \beta}$ 和 $g_2^{\tau_X}$ 决定的，只有这些元素必须在所有证明间保持一致，才能对 K 个等式做有意义的随机线性组合。一种朴素的做法是在所有证明组开始计算前，直接从公共输入派生统一的挑战点。然而这将导致证明者在生成承诺之前即可预知挑战值，从而能够针对性地构造满足验证等式但不对应有效见证的恶意多项式，从而破坏知识可靠性。为此，本方案设计了两轮交互协商机制以安全地派生共享挑战。第一轮中，各证明组首先独立完成承诺阶段并将承诺提交至协商主节点，协商主节点将所有证明组的承诺作为 Fiat-Shamir 变换的输入，派生共享挑战 α 并广播至所有证明组。第二轮中，各证明组以共享的 α 完成 X 轴聚合计算并将结果提交，协商主节点再次绑定所有 X 轴聚合结果派生共享挑战 β 并广播。由于两轮共享挑战的派生均绑定了所有证明组已提交的承诺信息，任何证明组都无法在提交承诺之前预知挑战值，从而在保证挑战点统一性的同时维持了知识可靠性。共享挑战确立后，协商主节点由 Fiat-Shamir 变换对全部压缩证明派生随机聚合系数 $\{r_k\}_{k=0}^{K-1}$ ，并从各压缩证明 $\pi_c^{(k)}$ 中的承诺和求值重算 $C_1^{(k)}, C_2^{(k)}$ ，随后对 K 个压缩证明的 X 轴商承诺、Y 轴聚合开启证明及相关线性组合量分别做随机线性组合，得到批量验证所需的聚合量。证明聚合的完整流程见算法 2。

算法 2: 证明聚合 ProofAggregate

输入: K 个证明组 $\{\mathcal{P}^{(k)}\}_{k=0}^{K-1}$ ，每组持有相同源链至目的链的子电路实例

输出: 批量证明 π_{batch}

// 各证明组 $\mathcal{P}^{(k)}$ ($k=0, \dots, K-1$) 并行执行:

1: $\text{com}_{H_X}^{(k)} \leftarrow \text{ProofCompress.Commit}(\mathcal{P}^{(k)})$ ，将 $\text{com}_{H_X}^{(k)}$ 发送至协商主节点 $\mathcal{P}_0^{(0)}$

// 协商主节点 $\mathcal{P}_0^{(0)}$ 执行:

2: $\alpha \leftarrow H(pp, \text{com}_{H_X}^{(0)}, \dots, \text{com}_{H_X}^{(K-1)})$ ，将 α 广播至所有证明组

// 各证明组 $\mathcal{P}^{(k)}$ 并行执行:

3: $\nu^{(k)} \leftarrow H(\alpha, k)$ ， $(\text{com}_{Q_X}^{(k)}, \mathcal{V}_X^{(k)}) \leftarrow \text{ProofCompress.XAgg}(\mathcal{P}^{(k)}, \alpha, \nu^{(k)})$ ，将 $(\text{com}_{Q_X}^{(k)}, \mathcal{V}_X^{(k)})$ 发送至 $\mathcal{P}_0^{(0)}$

// 协商主节点 $\mathcal{P}_0^{(0)}$ 执行:

4: $\beta \leftarrow H(\alpha, \text{com}_{Q_X}^{(0)}, \mathcal{V}_X^{(0)}, \dots, \text{com}_{Q_X}^{(K-1)}, \mathcal{V}_X^{(K-1)})$ ，将 β 广播至所有证明组

// 各证明组 $\mathcal{P}^{(k)}$ 并行执行:

5: $\mu^{(k)} \leftarrow H(\beta, k)$ ， $\pi_c^{(k)} \leftarrow \text{ProofCompress.YAgg}(\mathcal{P}^{(k)}, \beta, \mu^{(k)})$ ，将 $\pi_c^{(k)}$ 发送至 $\mathcal{P}_0^{(0)}$

// 协商主节点 $\mathcal{P}_0^{(0)}$ 执行:

6: 返回 $\pi_{\text{batch}} \leftarrow \{\pi_c^{(k)}\}_{k=0}^{K-1}$

如算法 2 所示, 整个证明聚合流程包含挑战协商、分布式证明生成和批量证明输出三个阶段。首先, 各证明组独立完成承诺阶段并将承诺提交至协商主节点, 协商主节点将所有承诺作为 Fiat-Shamir 变换的输入派生共享挑战 α 并广播至所有证明组, 对应算法第 1-2 行。其次, 各证明组以共享 α 并行完成 X 轴聚合计算并提交结果, 协商主节点再次绑定所有 X 轴聚合结果派生共享挑战 β 并广播, 对应算法 3-4 行。随后, 各证明组在本地调用算法 1 的子例程完成 Y 轴聚合, 生成各自的压缩证明 $\pi_c^{(k)}$ 并提交至协商主节点, 对应算法第五行。最终, 协商主节点将 K 个压缩证明集合作为批量证明 π_{batch} 输出, 对应算法第六行。

最后, 批量证明 π_{batch} 的验证, 验证者首先通过 Fiat-Shamir 变换从所有承诺信息中重算共享挑战 α, β 、各证明组的局部随机数及聚合系数 $\{r_k\}_{k=0}^{K-1}$, 并对 K 个压缩证明逐一执行代数约束检查。随后, 验证者利用聚合系数 r_k 对 K 个压缩证明的商承诺、聚合开启证明及相关线性组合量进行加权求和, 构造聚合验证量, 代入单个四配对等式完成批量验证。由于配对运算次数固定为常数 4 次, 与证明数量 K 无关, 批量验证的主要开销来自 K 次椭圆曲线标量乘法的聚合量重建, 整体验证复杂度中配对开销降至 $O(1/K)$ 量级, 详细理论分析见 6.1 节。

4 安全分析

本节证明所提方案在引入证明压缩和证明聚合机制后, 仍满足 zk-SNARK 的三个核心安全性质: 完备性、知识可靠性和零知识性。我们在代数群模型和随机预言机模型下, 将所提方案的安全性归约到底层 Pianist 协议的安全性以及 q -DLOG 困难假设。

定理 1(完备性): 对于任意有效的电路-见证对 $\{C_i, w_i\}_{i=0}^{K-1}$ 满足 $C_i(w_i) = 1$, 诚实证明者生成的批量证明 π_{batch} 能以概率 1 通过验证者的验证。

证明: 由 Pianist 协议的完备性, 诚实生成的见证多项式、置换多项式和商多项式满足聚合约束等式 $G(Y, X) + \lambda P_0(Y, X) + \lambda^2 P_1(Y, X) = V_x(X)H_x(Y, X) + V_y(Y)H_y(Y, X)$, 故代数检查必然通过。在证明压缩阶段, X 轴聚合商多项式 $Q_x(Y, X)$ 和 Y 轴聚合商多项式 $Q_y(Y)$ 由多项式除法构造, 其在 $\mathbb{F}[Y, X]$ 中的代数恒等性保证了 DKZG 承诺方案的开启证明在所有挑战点上正确, 从而合并后的配对等式 $e(\text{com}_{Q_x}, [Z_T(\tau_x)]_2) \cdot e(\pi_{1, \text{agg}}, g_2^{\tau_y - \beta})^\rho = e(D_{\text{lin}}, g_2) \cdot e(D_r, g_2^{\tau_x})$ 成立。在批量聚合阶段, K 个证明共享挑战点 α, β , 由双线性配对的线性性质, K 个独立成立的配对等式经随机系数 $\{r_k\}$ 的线性组合后必然成立, 故聚合后的配对验证等式同样成立。综上, 诚实生成的证明能够以概率 1 通过所有验证检查, 方案满足完备性。□

定理 2(知识可靠性): 在代数群模型和随机预言机模型下, 假设 q -DLOG 困难假设成立, 对于任意概率多项式时间敌手 $\mathcal{A}$, 若其能以不可忽略概率生成通过验证的批量证明 π_{batch} 但不持有有效见证, 则存在概率多项式时间提取算法 $\mathcal{E}$ 能从 $\mathcal{A}$ 输出的证明中提取出有效见证, 或破解 q -DLOG 困难假设。具体地, 敌手成功伪造证明的概率满足 $\Pr[\mathcal{A} \text{ succeeds}] \leq \epsilon_{\text{DLOG}} + (K+1)d/|\mathbb{F}|$, 其中 ϵ_{DLOG} 为破解 q -DLOG 的优势, d 为多项式最高次数, K 为批量聚合的证明数量。

证明: 我们通过定义一系列实验, 将敌手成功伪造证明的事件归约到 Schwartz-Zippel 引理失败或 q -DLOG 假设被破解。Game₀ 为真实实验, 敌手 $\mathcal{A}$ 输出通过验证的批量证明 π_{batch} 。Game₁ 与 Game₀ 一致, 但模拟器记录挑战协商阶段派生的所有共享挑战 α, β 和聚合随机数 ν, μ , 由于 Fiat-Shamir 变换在随机预言机模型下绑定了所有承诺信息, $\mathcal{A}$ 无法在生成承诺前预知挑战点, 故 $|\Pr[\text{Game}_1] - \Pr[\text{Game}_0]| = 0$ 。Game₂ 与 Game₁ 一致, 但若存在某个证明 k^* 使其 X 轴或 Y 轴聚合商多项式不满足构造定义, 实验中止。由 Schwartz-Zippel 引理, 该事件发生概率至多为 $2d/|\mathbb{F}|$, 故 $|\Pr[\text{Game}_2] - \Pr[\text{Game}_1]| \leq 2d/|\mathbb{F}|$ 。Game₃ 进一步要求若 K 个证明的随机线性组合等式成立但存在至少一个证明的原始等式不成立则中止, 由于聚合系数 $\{r_k\}$ 由 Fiat-Shamir 变换从所有压缩证明派生, 在随机预言机模型下不可预测, 根据 Schwartz-Zippel 引理该事件概率至多为 $K/|\mathbb{F}|$ 。经过上述实验序列后, $\mathcal{A}$ 必须为每个证明提供满足 Pianist 原始约束等式的多

项式，在代数群模型下，提取算法 $\mathcal{E}$ 可从 $\mathcal{A}$ 给出的群元素表示中提取出底层多项式系数从而获得有效见证，若提取失败则可构造 q -DLOG 问题的解，该事件概率至多为 ϵ_{DLOG} 。综合各步骤的概率界，有 $\Pr[\mathcal{A} \text{ succeeds}] \leq \epsilon_{\text{DLOG}} + (K+1)d/|\mathbb{F}|$ ，该值在 $|\mathbb{F}|$ 足够大时可忽略，故方案满足知识可靠性。□

定理 3(零知识性): 在随机预言机模型下，存在概率多项式时间模拟器 $\mathcal{S}$ ，对于任意概率多项式时间区分器 $\mathcal{D}$ ，模拟器生成的证明视图与真实证明者-验证者交互的视图在计算上不可区分，即 $|\Pr[\mathcal{D}(\text{Real})=1] - \Pr[\mathcal{D}(\text{Sim})=1]| \leq \text{negl}(\lambda)$ ，其中 λ 为安全参数。

证明: 设 Pianist 协议的零知识模拟器为 $\mathcal{S}_p$ ，我们基于 $\mathcal{S}_p$ 构造批量证明的模拟器 $\mathcal{S}$ 。 $\mathcal{S}$ 在不知道见证的情况下，首先随机选取共享挑战点 $\alpha, \beta \in \mathbb{F}$ 、聚合随机数 $\nu, \mu, \{r_k\} \in \mathbb{F}$ 以及多项式在挑战点的所有取值，然后利用陷门 τ_x, τ_y 反推计算满足配对等式的承诺 $\text{com}_{Q_x} = g^{Q_x(\tau_y, \tau_x)}$ 和聚合开启证明 $\pi_{1,\text{agg}}$ ，其中 $Q_x(Y, X)$ 由随机选取的取值反推构造。 $\mathcal{S}$ 通过编程随机预言机使得 Fiat-Shamir 变换的输出与预先选取的挑战值一致，从而确保模拟器生成的承诺和开启证明在分布上与真实视图保持一致。对于批量证明的模拟， $\mathcal{S}$ 独立调用上述过程生成 K 个压缩证明，并按照协议流程计算聚合量 $\text{com}_{Q_x, \text{total}}, \pi_{1, \text{total}}, D_{Y, \text{total}}$ 。由 DKZG 承诺方案的隐藏性，随机选取的多项式取值与真实多项式求值在计算上不可区分。由随机预言机的可编程性，Fiat-Shamir 挑战在两个视图中的分布完全一致。由于聚合操作仅涉及公开承诺和开启证明的线性组合，不引入额外的见证信息，故聚合过程保持零知识性。因此， $\mathcal{S}$ 生成的批量证明视图与真实视图在计算上不可区分，方案满足零知识性。□

5 性能分析

5.1 理论分析

本节从计算复杂度、通信复杂度和存储开销三个维度分析证明压缩和证明聚合机制带来的性能影响，并与原始 Pianist 协议进行对比。

计算复杂度方面，在证明压缩方案中，每个子节点除执行 Pianist 原有的拉格朗日插值、FFT 和 MSM 运算外，还需额外计算 X 轴聚合商多项式 $Q_{X,i}(X)$ 及其承诺 $\text{com}_{Q_{X,i}}$ ，该步骤涉及常数个多项式的线性组合与除法运算以及一次 MSM，引入的额外开销为 $O(T)\mathbb{F} + O(T)\mathbb{G}$ ，相对于 Pianist 的 $O(T \log T)\mathbb{F} + O(T)\mathbb{G}$ 在渐近意义上可忽略，故子节点的总计算复杂度同样为 $O(T \log T)\mathbb{F} + O(T)\mathbb{G}$ 。主节点除聚合多项式承诺外，还需聚合 X 轴商承诺 com_{Q_x} 并计算 Y 轴聚合多项式 $G_y(Y)$ 与商多项式 $Q_y(Y)$ ，引入 $O(M \log M)\mathbb{F} + O(M)\mathbb{G}$ 的额外开销。但由于实际部署中 $M \ll T$ ，主节点的额外开销相对于子节点的计算量可忽略。在证明聚合方案中，协商主节点需对 K 个独立证明执行挑战协商和承诺聚合，挑战协商通过 Fiat-Shamir 变换派生共享挑战，代价为 $O(K)\mathbb{H}$ 。承诺聚合需计算聚合系数和聚合承诺 $\text{com}_{Q_x, \text{total}}, \pi_{1, \text{total}}$ 等，涉及 K 次椭圆曲线标量乘法，代价为 $O(K)\mathbb{G}$ ，与批量大小 K 呈线性关系。在验证复杂度方面，单个压缩证明的验证复杂度为 $O(1)\mathbb{P} + O(1)\mathbb{G} + O(1)\mathbb{F}$ ，通过 X 轴和 Y 轴的多点聚合以及配对合并，本方案将验证复杂度保持在常数级，未引入额外的渐近开销。批量验证 K 个证明的总复杂度为 $O(K)\mathbb{G} + O(K)\mathbb{F} + O(1)\mathbb{P}$ ，其中 $O(K)$ 的开销来自 Fiat-Shamir 挑战重算、代数约束检查和聚合承诺重建，而配对运算次数被压缩至常数 4 次。由于配对运算的耗时通常是椭圆曲线标量乘法的 10-20 倍，将配对次数从 $O(K)$ 降至 $O(1)$ 带来的性能提升十分显著，当 K 较大时批量验证的平均每证明配对开销趋近于 $O(1/K)\mathbb{P}$ ，相比单独验证可节省约 $(K-1)/K$ 的配对开销。

在通信复杂度方面，在证明压缩方案中，单个子节点还需额外发送 X 轴聚合商承诺，仅增加了一个 $\mathbb{G}_1$ 元素，因此总通信量仍为 $O(M)|\mathbb{G}|$ 。在证明聚合方案中，挑战协商需要主节点收集 K 个证明的承诺和 X 轴聚合结果，通信量为 $O(K)|\mathbb{G}|$ ，与批量大小线性相关。以 BN254 曲线、 $K=10$ 为例，挑战协商的总通信量约为 640 字节，远小于子节点到主节点的通信量，在实际部署中可接受。

在证明大小方面，Pianist 协议继承了 Plonk 的随机线性组合优化，但由于置换约束的验证需要在两个不同的 X 轴挑战点 α 和 $\omega_x \alpha$ 处打开置换多项式，无法将这两个点的开启证明聚合，故 X 轴维度需保留

两个独立的开启证明。本方案通过多点聚合技术将针对不同挑战点的 X 轴开启证明聚合为单个商承诺，最终将 Y 轴的多个多项式开启证明聚合为单个 $\pi_{1,agg}$ ，从而将单个压缩证明的大小压缩至约 $12|\mathbb{G}_1| + 7|\mathbb{F}|$ ，相较于 Pianist 减少约 50%，且与电路规模 N 和子节点数量 M 无关。批量证明的大小为 $(11K+2)|\mathbb{G}_1| + 7K|\mathbb{F}|$ ，虽与批量大小 K 呈线性关系，但聚合开启证明部分仅为常数大小。以 BN254 曲线和 $K=10$ 为例，批量证明大小约为 5824 字节。批量聚合机制的主要优势在于验证复杂度的降低而非证明大小的压缩。

5.2 验证实验

实验对比了本文方案与 Pianist 协议在证明大小、证明验证时间及链上 Gas 开销三个维度上的性能差异。本文的代码实现与 Pianist 协议同样基于 gnark 和 gnark-crypto 库实现。实验平台运行 Debian Linux 6.6.87.2，处理器为 Intel Core i5-12400F，内存容量 32 GB，存储为 1 TB 固态硬盘。实验所用电路为 Keccak-256 哈希函数电路，用于验证对 64 字节消息的 Keccak-256 哈希计算的正确性。Keccak-256 是以太坊中用于 MPT 状态树节点哈希的核心算法，选取该电路能够真实反映跨链状态证明场景下的典型计算规模。电路采用位级门实现，对 Keccak-f[1600] 置换的 24 轮迭代展开，生成约三十万个 Plonk 门约束。系统部署两个子节点，总电路规模 2^{19} ，每个子节点独立持有不同 64 字节消息并行完成本地子电路证明。

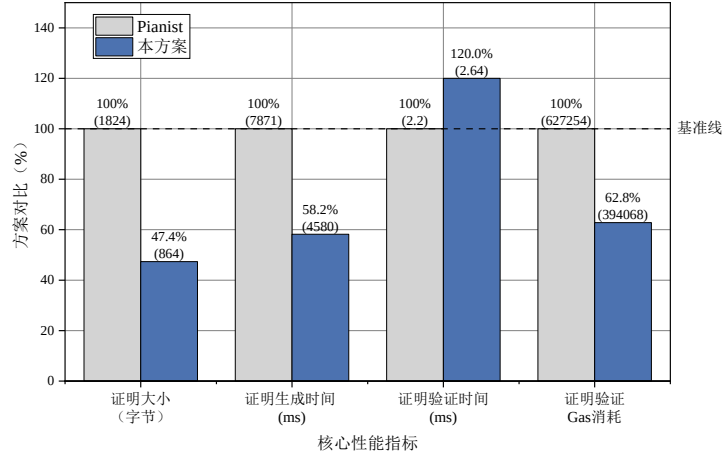


图 1 证明压缩的性能对比

我们将证明压缩和原 Pianist 方案在证明大小、证明生成时间、验证时间和链上 Gas 开销四个维度进行了对比，如图 1 所示。证明大小方面，本方案的压缩证明大小为 864 字节，较 Pianist 的 1824 字节减少 52.6%。压缩收益来源于两方面，在 X 轴通过多点聚合将针对 α 和 $\omega_x \alpha$ 两个挑战点的多个多项式开启证明聚合为单一商承诺 com_{Q_x} ，在 Y 轴通过聚合将 7 个独立的一元多项式开启证明折叠为单个 $\pi_{1,agg}$ ，从而使上链证明从约 27 个 $\mathbb{G}_1$ 元素降至约 12 个。此实验结果与理论推导完全吻合。证明生成时间方面，本方案的压缩证明生成时间为 4580 毫秒，较 Pianist 的 7871 毫秒减少 41.8%。这一加速来自证明压缩阶段对多项式求值过程的优化，X 轴聚合商多项式的计算复用了子节点已有的多项式求值结果，Y 轴折叠利用 FFT 在规模 M 的小规模多项式上完成，额外计算开销可忽略不计。实验结果与理论分析一致，证明压缩在渐近复杂度上与原始 Pianist 保持相同量级 $O(T \log T)\mathbb{F} + O(T)\mathbb{G}$ ，加速来源于 Shplonk 将多次独立 KZG 开启证明的 MSM 运算归约为单次所带来的常数因子改善。链上 Gas 开销方面，本方案单个证明的验证消耗 394068 Gas，较 Pianist 的 627254 Gas 节省 37.2%。节省主要来自 solidity 合约调用中 calldata 的规模缩减。验证时间方面，本文方案证明验证耗时 2.641 毫秒，相较于 Pianist 的 2.201 毫秒多出约 20%。该差异来自于本方案在验证端需要额外计算两个 $\mathbb{G}_2$ 标量乘法，而 Pianist 直接从 SRS 中读取对应 $\mathbb{G}_2$ 元素。

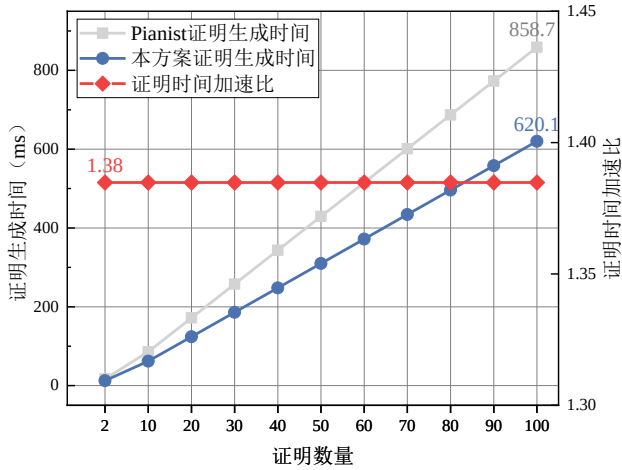


图 2 证明生成时间对比

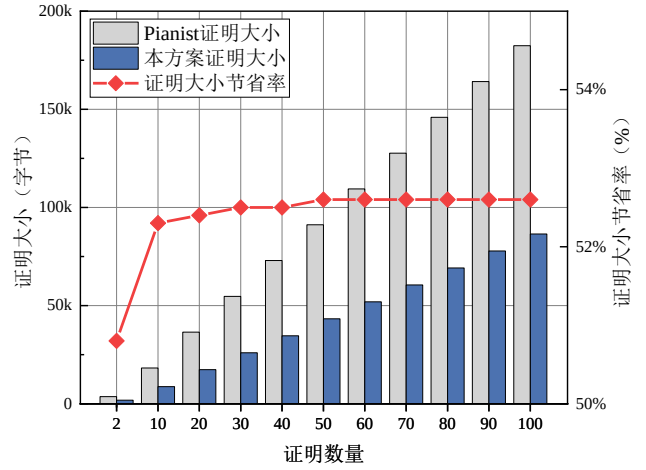


图 3 证明大小对比

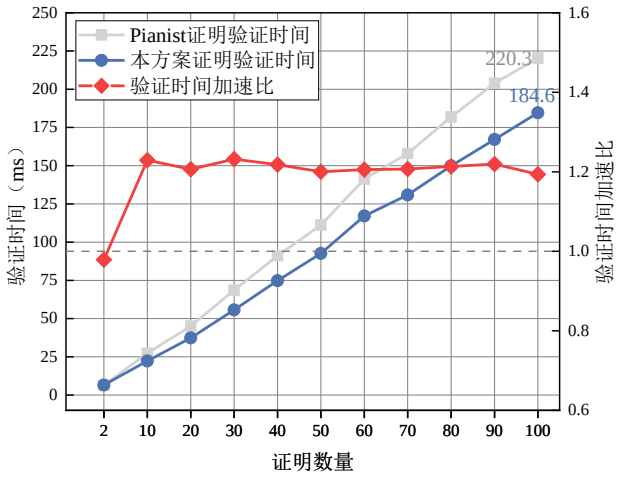


图 4 证明验证时间对比

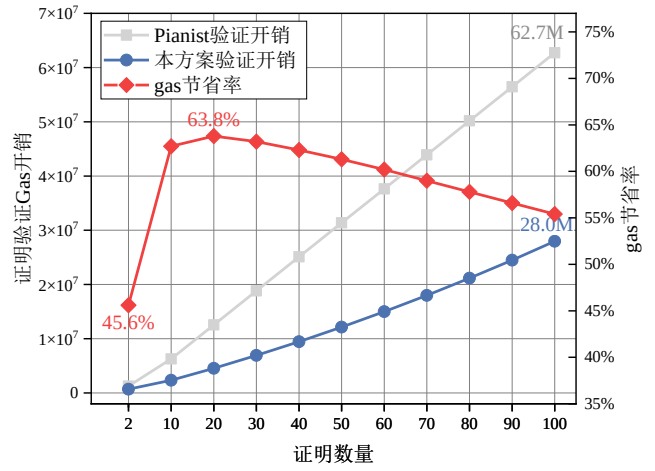


图 5 证明验证 Gas 开销对比

证明批量聚合部分，我们分别使用本方案和 Pianist 协议生成了 K 个证明，在证明生成时间，证明大小、证明验证时间和证明链上验证开销四个方面进行了对比。

(1). 证明生成时间随证明数量 K 的变化如图 2 所示，由于 K 个证明相互独立并行生成，总时间与 K 呈线性关系，本方案相对 Pianist 的生成时间加速比保持恒定为 1.38 倍，这与之前的理论分析一致，加速来源于 X 轴 Shplonk 聚合，在 Pianist 协议中，每个子节点需为 13 个多项式在 α 处及 $Z(Y, X)$ 在 $\omega_x \alpha$ 处共 14 个 KZG 开启各自计算一次 MSM，本方案将这 14 个开启合并为单个 Shplonk 商多项式 $Q_{x,i}(X)$ ，每个子节点只需 1 次 MSM，减少约 93% 的子节点 MSM 运算量，但由于 FFT 等其他开销占总时间的比重，整体加速比为 1.38 倍。

(2). 证明大小随证明数量 K 的变化如图 3 所示，两个方案均随 K 线性增长，本方案相比于 Pianist 节省率约为 52.5%。实验数据理论分析一致，本方案批量大小为 $864 \cdot K + 68$ 字节，Pianist 协议为 $1824 \cdot K$ 字节，节省率趋近于压缩率 $(1824 - 864) / 1824 \approx 52.6\%$ 。

(3). 验证时间随证明数量 K 的变化如图 4 所示。当 $K \geq 20$ 时，加速比持续超过 1.15，并在 $K=20$ 和 $K=40$ 时达到峰值约 1.24，此后随 K 增大略有回落。加速的根本原因在于本方案的聚合验证将配对次数维持在常数 4 次，而 Pianist 的逐一验证需执行 $4K$ 次配对。随着 K 增大，配对开销在总验证时间中的占比上升，常数配对的优势逐渐显现。上述趋势与理论分析相符，批量验证复杂度 $O(K)G + O(K)F + O(1)P$ 中，配对项的摊销代价随 K 增大而递减；加速比在 $K \approx 20$ 后趋于平稳而非持续增长，这是因为 $O(K)G$ 的 MSM 计算开销逐渐主导验证耗时所致。

(4). 链上验证的 Gas 开销随证明数量 K 的变化如图 5 所示。Pianist 协议的链上证明验证 Gas 开销与 K 完全线性相关，每新增一个证明即需独立调用一次验证合约，斜率为 627254 Gas。本方案的 Gas 增长斜率显著更小，且节省率在小 K 时随 K 快速上升，当 $K=1$ 时节省 23.1%， $K=10$ 时节省 62.7%， $K=20$ 时达到峰值 63.8%。此后节省率缓慢回落， $K=100$ 时仍维持 55.4%。峰值节省出现在 $K \approx 20$ 处，与理论分析的综合预测一致，本方案的证明链上验证总 Gas 开销由常数次配对验证开销和正比于 $864K$ 的线性 calldata 开销两部分叠加而成，随着 K 的增大，配对固定成本的摊薄效应先占主导，节省率持续上升，而后 calldata 线性项逐渐主导，节省率缓降并趋近于 calldata 纯压缩率 52.6%。

6 结束语

本文针对分布式零知识证明在区块链跨链状态证明场景中验证开销过高的问题，在 Pianist 协议的基础上提出了本方案。方案通过两层优化实现了证明的高效验证，在单证明层面，基于多点聚合技术将分布式生成的多个多项式开启证明压缩为常数大小，使证明大小减少约 52.6%，子节点证明生成时间加速约 1.38 倍。在批量验证层面，利用 KZG 承诺的加法同态性通过共享挑战点协议将 K 个证明聚合为单个批量证明，将链上配对运算次数从 $O(K)$ 降至常数 4 次，在 $K \in [10, 50]$ 的典型场景下可节省超过 60% 的链上 Gas 开销。安全性分析表明，方案在代数群模型和随机预言机模型下不引入额外的安全假设，完备性、知识可靠性和零知识性均得到保障。理论分析与实验结果相互印证，验证了方案的正确性和实用性。然而，本方案仍存在若干局限性有待后续研究解决。一方面，证明压缩引入的 Y 轴折叠步骤由主节点集中执行，主节点在证明生成过程中承担较多协商工作，在子节点数量 M 极大时可能成为性能瓶颈；另一方面，批量聚合机制要求 K 个证明共享相同的挑战点 α 和 β ，需要额外的两轮协商通信，在跨机构分布式场景中增加了部署复杂度。此外，本方案目前基于可信设置，如何在透明设置下实现同等的验证效率，也是值得深入探索的方向。

参考文献

- [1] YANG Ningbin, TANG Chunming, ZHOU Quan, *et al.* Dynamic consensus committee-based for secure data sharing with authorized multi-receiver searchable encryption[J]. IEEE Transactions on Information Forensics and Security, 2023, 18: 5186 - 5199.
- [2] YIN Zeyuan, ZHANG Bingsheng, XU Jingzhong, *et al.* Bool network: An open, distributed, secure cross-chain notary platform[J]. IEEE Transactions on Information Forensics and Security, 2022, 17: 3465 - 3478.
- [3] LIANG Xinyu, CHEN Jing, DU Ruiying. XPull: A Relay-Based Blockchain Intercommunication Framework Achieving Cross-Chain State Pulling[J]. Chinese Journal of Electronics, 2024, 33(5): 1261-1273.
- [4] LI Ming, WENG Jian, WENG Jiasi, *et al.* IvyCross: A Privacy-Preserving and Concurrency Control Framework for Blockchain Interoperability[J]. IEEE Transactions on Mobile Computing, 2025, 24(10): 9334 - 9351.
- [5] GROTH J. On the size of pairing-based non-interactive arguments[C], Annual International Conference on the Theory and Applications of Cryptographic Techniques, "Vienna, Austria", 2016: 305-326.
- [6] GABIZON A, WILLIAMSON ZJ, CIOBOTARU O. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge[C], Cryptology ePrint Archive, "Online", 2019.
- [7] BUTERIN V. An Incomplete Guide to Rollups [OL]. <https://vitalik.ca/general/2021/01/05/rollup.html>, 2021.1.
- [8] SASSON E, CHIESA A, GARMAN C, *et al.* Zerocash: Decentralized anonymous payments from bitcoin[C], 2014 IEEE Symposium on Security and Privacy, "Berkeley, USA", 2014: 459-474.
- [9] WU H, ZHENG Wenting, CHIESA A, *et al.* DIZK: A distributed zero knowledge proof system[C], 27th USENIX Security Symposium (USENIX Security 18), "Baltimore, USA", 2018: 675-692.
- [10] XIE Tiancheng, ZHANG Jiaheng, CHENG Zerui, *et al.* zkbridge: Trustless cross-chain bridges made practical[C], Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, "Los Angeles, USA", 2022: 3003-3017.
- [11] ZHANG, Jiaheng, XIE Tiancheng, ZHANG Yupeng, *et al.* Transparent polynomial delegation and its applications to zero knowledge proof[C], 2020 IEEE Symposium on Security and Privacy (SP), "San Francisco, USA", 2020: 859-876.
- [12] LIU Tianyi, XIE Tiancheng, ZHANG Jiaheng, *et al.* Pianist: Scalable zkrollups via fully distributed zero-knowledge proofs[C], 2024

- IEEE Symposium on Security and Privacy (SP), "San Francisco, USA", 2024: 1777-1793.
- [13] KATE A, ZAVERUCHA G M, GOLDBERG I. Constant-size commitments to polynomials and their applications[C], International Conference on the Theory and Application of Cryptology and Information Security, "Singapore, Singapore", 2010: 177-194.
 - [14] ROSENBERG M, MOPURI T, HAFEZI H, *et al.* Hekaton: Horizontally-scalable zkSNARKs via proof aggregation[C], Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, "Salt Lake City, USA", 2024: 929-940.
 - [15] LI Chongrong, ZHU Pengfei, LI Yun, *et al.* HyperPianist: Pianist with Linear-Time Prover and Logarithmic Communication Cost[C], 2025 IEEE Symposium on Security and Privacy (SP), "San Francisco, USA", 2025: 3383-3401.
 - [16] CHEN Binyi, BÜNZ B, BONEH D, *et al.* Hyperplonk: Plonk with linear-time prover and high-degree custom gates[C]. Annual International Conference on the Theory and Applications of Cryptographic Techniques, 2023: 499-530.
 - [17] BOWE S, GRIGG J, HOPWOOD D. Recursive proof composition without a trusted setup[J]. Cryptology ePrint Archive, 2019.
 - [18] GAILLY N, MALLER M, NITULESCU A. Snarkpack: Practical snark aggregation[C]. International Conference on Financial Cryptography and Data Security. Cham: Springer International Publishing, 2022: 203-229.
 - [19] BONEH D, DRAKE J, FISCH B, *et al.* Efficient polynomial commitment schemes for multiple points and polynomials[OL]. <https://eprint.iacr.org/2020/081>.
 - [20] ZIPPEL R. Probabilistic algorithms for sparse polynomials[C], International symposium on symbolic and algebraic manipulation, "Marseille, France", 1979: 216-226.
 - [21] SCHWARTZ JT. Fast probabilistic algorithms for verification of polynomial identities[J]. Journal of the ACM (JACM), 1980, 27(4): 701-717.

孙文拓：男，博士生，研究方向为数据安全与区块链。

薛开平：男，教授，研究方向为下一代网络体系结构与网络安全

An Efficient Verification Scheme for Cross-Chain State Proofs

Wentuo Sun^①

①(School of Cyber Science and Technology, University of Science and Technology of China, Hefei 230026, China)

Abstract:

Objective With the rapid development of blockchain technology, both public chains and consortium/private chains have been widely deployed across various domains for data management and endorsement. However, these systems differ in architecture and lack mutual trust, which hinders secure and efficient cross-domain data circulation and limits the value of data. Cross-chain state proof based on zero-knowledge Succinct Non-interactive Argument of Knowledge (zk-SNARK) has emerged as a promising solution due to its trustless nature and succinct verification. To address the prover bottleneck in large-scale cross-chain state proof circuits, distributed zero-knowledge proof techniques have been proposed, partitioning circuits and distributing computation across multiple worker nodes. However, existing distributed schemes primarily focus on optimizing prover efficiency and reducing inter-node communication, while paying insufficient attention to on-chain verification overhead. Since blockchain storage and computational resources are scarce and every cryptographic operation incurs significant gas cost, high verification overhead remains a critical obstacle to deploying distributed zero-knowledge proofs in real-world cross-chain scenarios. This paper proposes a distributed zero-knowledge proof scheme with efficient verification, which significantly reduces on-chain verification cost without sacrificing prover efficiency or communication complexity.

Methods The proposed scheme builds upon the Pianist protocol, a distributed zk-SNARK system based on Plonk with bivariate polynomial constraints. Two layers of optimization are introduced. First, a distributed polynomial aggregation algorithm is designed to compress individual proofs. Inspired by Shplonk's multi-point aggregation technique, the algorithm aggregates the X-axis opening proofs at challenge points α and $\omega_x \alpha$

into a single quotient commitment, and further folds the multiple Y-axis opening proofs into one through random linear combination. Worker nodes compute partial aggregation quotient polynomials locally and in parallel, while the master node performs global aggregation. The X-axis and Y-axis pairing verification equations are then merged via random linear combination, reducing the per-proof pairing count and laying the foundation for batch aggregation. Second, leveraging the additive homomorphism of KZG commitments, a proof batch aggregation mechanism is designed to combine K independently generated compressed proofs into a single batch proof. A two-round challenge negotiation protocol ensures that all K proofs share identical challenge points α and β , with shared challenges derived via Fiat-Shamir transformation binding to all commitment information to preserve knowledge soundness. The K pairing verification equations are then aggregated into one through random linear combination, reducing the on-chain pairing count from $O(K)$ to constant.

Results and Discussions Theoretical and experimental analyses demonstrate the effectiveness of the proposed scheme. Theoretically, the prover time complexity remains $O(T \log T)\mathbb{F} + O(T)\mathbb{G}$ per worker node and the per-node communication remains $O(M)|\mathbb{G}|$, both consistent with the original Pianist protocol. The single compressed proof maintains constant verification complexity $O(1)\mathbb{P} + O(1)\mathbb{G} + O(1)\mathbb{F}$, while batch verification of K proofs achieves $O(K)\mathbb{G} + O(K)\mathbb{F} + O(1)\mathbb{P}$, with the pairing count compressed to a constant of 4 regardless of K . Security analysis under the Algebraic Group Model and Random Oracle Model proves that the scheme satisfies completeness, knowledge soundness, and zero-knowledge properties, with adversarial advantage bounded by $\epsilon_{\text{DLOG}} + (K+1)d/|\mathbb{F}|$, introducing no additional security assumptions beyond those of the underlying Pianist protocol. Experiments based on the Keccak-256 hash circuit (approximately 300,000 Plonk gates) demonstrate that the compressed proof size is 864 bytes, a 52.6% reduction compared to Pianist's 1824 bytes, while the on-chain verification gas cost drops from 627,254 to 394,068, saving 37.2% per proof. For batch verification, the gas saving rate peaks at 63.8% when $K=20$ and remains above 55% even at $K=100$. The proof generation time is also accelerated by approximately $1.38\times$ due to the reduced number of MSM operations per worker node.

Conclusions This paper presents a novel distributed zero-knowledge proof scheme for the efficient verification of cross-chain state proofs. By introducing proof compression based on Shplonk multi-point aggregation and proof batch aggregation leveraging KZG commitment homomorphism, the scheme significantly reduces on-chain verification cost while preserving the linear scalability and low communication overhead of the underlying Pianist protocol. Theoretical analysis and experimental results jointly verify the correctness and practicality of the scheme. Several limitations remain to be addressed in future work. First, the Y-axis aggregation step in proof compression is centrally executed by the master node, which may become a performance bottleneck when the number of worker nodes M is extremely large. Second, the batch aggregation mechanism requires K proofs to share identical challenge points, introducing two additional rounds of coordination that increase deployment complexity in distributed cross-institutional scenarios. Third, the current scheme is built upon a trusted setup; achieving comparable verification efficiency under a transparent setup, as well as extending the scheme to support recursive proof composition, are promising directions for further research.

Key words: Blockchain; Cross-chain; Zero-knowledge proof; Distributed zero-knowledge proof; Proof aggregation